Ders 3 Genişletilmiş Notu: Factory Method, Builder, Prototype ve Singleton Kalıpları

Ders 3 Genişletilmiş Notu: Factory Method, Builder, Prototype ve Singleton Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Abstract Factory ile Factory Method Arasındaki Fark
- 2. Factory Method Kalıbı
- 3. Factory Method Örneği: E-Ticaret Sitesi
- 4. Builder Kalıbı
- 5. Builder Örneği: Araç Adlarını Çözümleyen Modül
- 6. Prototype Kalıbı
- 7. Prototype Örneği: Oyun İçin Bölüm Düzenleyicisi
- 8. Singleton Kalıbı
- Örnek Sorular
- Dersin Toparlaması

1. Abstract Factory ile Factory Method Arasındaki Fark

Ders, geçen hafta görülen Abstract Factory kalıbının kısa bir hatırlatmasıyla başladı. O kalıpta örnek problem üç farklı biçimde çözülmüştü: akla ilk gelen ama bakımı zorlaştıran, gereksinim değişikliklerine dayanıksız bir çözüm; nesneye dayalı ilkelerle uyumun ve bağlaşımın iyileştiği, eklemelerin kolaylaştığı bir çözüm; son olarak kalıbın uygulandığı ve neyin kazanılıp neyin kaybedildiğinin tartışıldığı çözüm. Sıradaki kalıp Factory Method'dur.

Hoca iki kalıbın adında da "Factory" geçtiği için sık karıştırıldığını, ama aslında birbirinden çok farklı kalıplar olduğunu vurguladı. "Factory" bir şeyin, yani belirli bir nesnenin üretilmesi anlamına gelir. Abstract Factory'de bir ürün ailesinden nesne üretilir ve istemci herhangi bir ürün ailesiyle çalışabilir. Factory Method ise tek bir nesne oluşturmak için kurulmuş bir kalıptır. İkisi arasında küçük bir paralellik vardır: Factory Method'da da nesnenin sınıfına, arayüzü sağlayan sınıfın alt sınıfları karar verir. Ama çözdükleri problemler belirli bir noktada farklıdır.

2. Factory Method Kalıbı

2.1 Örnek sorun: belge uygulamaları için çerçeve program

Farklı belge türleriyle çalışabilecek uygulamalar geliştirmeyi sağlayan bir çerçeve program (framework) yazıyoruz. İki temel varlık vardır: üzerinde çalışılacak belgeler ve hazırlanacak uygulama. Çerçeveyi kullanan programcılar, bu iki temel sınıftan kalıtımla kendi alt sınıflarını üretip kendi yazılımlarını hazırlayacaktır.

Problem şudur: bir belge oluşturmak gerektiğinde çerçeve program bunu nasıl yapacak? Belge türünden bağımsız bazı işler vardır, ama bunlar da belge türünden yoğun biçimde etkilenebilir. Çerçevedeki uygulama sınıfı, kullanıcı programcının türetebileceği çok farklı belge sınıfları hakkında önceden bilgi sahibi olamaz; çerçeve de olabildiğince geniş çaplı olmak ister.

2.2 Örnek çözüm

Hoca slayttaki şemayı yatay bir çizgiyle ikiye böldü (ekrandan, 00:20:00): üst kısım bizim yazdığımız çerçeve, alt kısım bu çerçeveyi kullanarak geliştirilecek uygulamalardır. Şema metne dökülünce:

```
Çerçeve (framework):
  Document (soyut)                 Application (soyut)
    Open()      {soyut}              CreateDocument()  {soyut}
    Close()     {soyut}              NewDocument()
    Save()                           OpenDocument()
    Revert()
  Application --docs--> Document  (bir uygulama birden fazla belgeyle çalışır)

  NewDocument() notu:
    Document* doc = CreateDocument();
    docs.Add(doc);
    doc->Open();

Uygulama:
  MyDocument  --kalıtım-->  Document
  MyApplication --kalıtım--> Application
    CreateDocument()   notu: return new MyDocument
  MyApplication - - -> MyDocument  (bağımlılık)
```

Bir uygulama birden fazla belgeyle çalışır ve aynı anda birden fazlası açık olabilir. Sınıf adlarının italik yazılması soyut sınıf anlamına gelir.

`Document` soyut sınıfında hem soyut hem somut metotlar bulunur. Her belge açılacak, kapanacak, kaydedilecek ve geri alınacaktır (Revert). Geri alma işlemleri daha sonra Memento kalıbıyla ayrıca modellenecektir; şimdiki odak bu değildir. Bir dosyayı açıp baytlarını okumak her dosya türünde yapılır, ama dosyanın mantıksal içeriğine göre ek işlemler gerekebilir ve bunlar çok farklı olabilir. Belgeyi kapatmak dosyanın fiziksel olarak kapatılmasıdır, ama öncesinde uygulamanın mantığına göre tutarlılık denetimi gibi farklı işler istenebilir. Kaydetme ise ikilik kipte bilgi aktarımıdır; bu somut bir metottur ve çerçeve onu kodlayarak kullanıcının işini kolaylaştırır.

`Application` soyut sınıfında da soyut ve somut metotlar vardır. Yeni belge oluşturmanın genel kısmı çerçevenin üstlenebileceği bir iştir: bir dosya kaydetme iletişim kutusu göstermek, çok sekmeli bir arayüz varsa yeni sekme, yoksa yeni pencerede boş bir tuval açmak. Bunlar kolay olsa da kullanıcı programcının üstüne yıkılmak istenmeyen temel işlerdir. Belgenin gerçekten oluşturulması ise belgenin türüne göre değişir ve çerçeveyi kodlayan kişi kullanıcılarının neler gerçeklemek isteyeceğini önceden bilemez. Bu yüzden soyut metotlar eklenir; alt sınıflarda programcı bunları geçersiz kılarak (override) kendi işini kodlar.

`NewDocument()` metodunun içi çerçevede örnek olarak verilmiştir: belge türünde bir işaretçi `CreateDocument()` çağrısının dönüşüyle doldurulur, açık belgelerin tutulduğu veri yapısına (bağlı liste, dizi listesi ya da bir tablo) eklenir ve belge açılır. `CreateDocument()` ise `Application`'ın alt sınıfı olan `MyApplication`'da geçersiz kılınır; içinde daha çok komut olabilir ama sonunda oluşturulan `MyDocument` nesnesi döndürülür.

Temel mantık şudur: yapılabilecek kısımları somut metotlarda yap; farklı uygulamalara bağlı kısımlar için soyut metotlar tanımla ve kodu bu soyut metotlar üzerinden tamamla. Soyut metotların gövdelerini alt sınıflarda kullanıcı programcı doldurur.

2.3 Güncel UML gösterimi üzerine hatırlatmalar

Hoca şema üzerinde iki gösterim konusunu hatırlattı. Birincisi, Dörtlü Çete eski UML gösterimini kullandığı için kalıtım okunu bazı şekillerde çizginin ortasına koymuştur; güncel standartta ok ucu üst sınıfa değecek biçimde çizilmelidir. İkincisi, yeni standartta soyut sınıflar yalnız italik yazılmaz; italikten bağımsız olarak sınıfın üst bölmesinde, adın üstünde «abstract» stereotipi de yazılır. Stereotip, iki küçüktür ve iki büyüktür işaretinin tek karakter hâlindeki biçimleri («») arasına yazılır.

2.4 Başka çözüm seçenekleri

Bu tür bir problemi çözmenin başka yolları da vardır. Örneğin Java'daki generic sınıf yapısıyla belge türü parametreye bağlanabilir. Bu bir noktaya kadar işi kolaylaştırır, ama dile özel bir çözümdür: başka dillerde generic sınıf desteği olmayabilir ya da dilin standart parçası olmayan ek kütüphanelerle geliyor olabilir; bu da taşınabilirliği azaltır. C++'taki template sınıflar da benzer işleri yapmayı sağlar. Derste kalıbın önerdiği çözüme odaklanıldı.

2.5 Kalıp yapısı

Kalıp terimleriyle konuşulursa uygulama sınıfı Creator'dır (ekrandan, 00:23:05):

```
Product (soyut)                    Creator (soyut)
   ^                                 FactoryMethod()  {soyut}
   |                                 AnOperation()    notu: ... product = FactoryMethod() ...
ConcreteProduct  <- - - - - - - -  ConcreteCreator  --kalıtım--> Creator
                                     FactoryMethod()  notu: return new ConcreteProduct
```

- Creator: ürün oluşturmak için bir oluşturucu metoda sahiptir, ama bu metot soyut tanımlanmıştır.
- ConcreteCreator: fabrika metodunu geçersiz kılarak somut ürünler oluşturur.
- Product ve ConcreteProduct: ortaya çıkan somut ürün istemci tarafından istenildiği gibi kullanılabilir.

Örnekteki Creator'dan Product'a (Application'dan Document'a) olan çoklu sahiplik ilişkisi isteğe bağlıdır; çözülen problem gerektirdiği için kurulmuştur, kalıbın gereği değildir. Kalıbı gerçeklemiş olmak için gereken şunlardır: soyut bir oluşturucu sınıf, onun soyut bir fabrika metodu ve o sınıfın içinde soyut metot cinsinden tanımlanan işlemleri çağıran en az bir somut metot.

2.6 Kalıbın uygulanabileceği anlar

- Bir sınıf yeni bir nesne oluşturmalıdır, ama nesnenin tam türü kodlama aşamasında belli değildir ya da dinamik olarak değiştirilmesi istenmektedir.
- Bir sınıf kendi sorumluluğunu birden fazla yardımcı alt sınıftan birine iletecektir ve hangi yardımcı alt sınıfın oluşturulacağı o andaki duruma bağlıdır.

2.7 İkinci örnek: çizim uygulamasında şekil manipülatörleri

Bir çizim uygulamasında (ya da bir uygulamanın çizim modülünde) eklenen her şeklin fareyle değiştirilmesi isteniyor (ekrandan, 00:27:20). Şema metne dökülünce:

```
Client  -->  Figure (soyut)            Client  -->  Manipulator (soyut)
               CreateManipulator() {soyut}             DownClick() {soyut}
               ...                                     Drag()      {soyut}
                                                       UpClick()   {soyut}
LineFigure  : Figure                   LineManipulator : Manipulator
  CreateManipulator()  - - - - - - ->    DownClick(), Drag(), UpClick()
TextFigure  : Figure                   TextManipulator : Manipulator
  CreateManipulator()  - - - - - - ->    DownClick(), Drag(), UpClick()
```

Çizimde çizgiler, metinler, dikdörtgenler, daireler, Bezier eğrileri gibi her türlü şekil olacaktır. Bir çizgi tıklandığında uzunluğu değiştirilebilir, döndürülebilir, kesik çizgi hâline getirilebilir ya da kalınlığı (örneğin 2, 3, 5) ayarlanabilir. Bu işlemler bir metin kutusu için çok anlamlı değildir; metin kutusu en ve boy olarak büyütülebilir, Shift'e basılı tutulursa en-boy oranı korunarak büyütülebilir, ama "kesikli metin" diye bir şey yoktur. Demek ki farklı şekil türleri farklı manipülatörlerle düzenlenmelidir; yine de bütün manipülatörlerin ortak bazı işlemleri olabilir.

Örnekte manipülatörün bütün metotları soyuttur. Bu durumda `Manipulator` bir Java arayüzü (interface) olarak da tanımlanabilir. Ama tek bir somut metot olacaksa ya da alt sınıflara aktarılması gereken bir alan varsa soyut sınıf tercih edilmelidir. Bütün bunları ortak bir çatı altında

toplamak için soyut üst sınıflar `Figure` ve `Manipulator` ile onların somut alt sınıfları kodlanıp ilişkilendirilir.

Bazı şekiller özel bir manipülatöre ihtiyaç duymuyorsa `Manipulator` sınıfını soyut değil somut yapıp varsayılan manipülatör olarak kullanmak düşünülebilir. Ancak bu kalıbın yapısını biraz fazla bozar. Daha çok tercih edilen yol, bir `DefaultManipulator` alt sınıfı oluşturup varsayılan manipülatörle idare edebilecek şekil türlerini onunla ilişkilendirmektir. Bu, daha az manipülatör kodlamak açısından kolaylık sağlar; ama bu kolaylık yalnız uygulamanın mantığı buna izin verdiği için vardır, yoksa her biri için ayrı manipülatör kodlanır.

Manipülatörden şekle doğrudan bir ilişki olmak zorunda değildir, ama şeklin kendi manipülatörünü bilmesi gerekebilir; ilişki iki yönlü de kurulabilir. Farklı şekiller ve manipülatörler arasında ortak yapılabilecek işler varsa, kalıbın örnek yapısında olduğu gibi soyut manipülatörden şekle bir ilişki kurulabilir. Ancak bu ilişki kalıbın bir gerekliliği değildir.

3. Factory Method Örneği: E-Ticaret Sitesi

3.1 Problem

Hoca örneği Head First Design Patterns kitabındaki bir örneği değiştirerek aldığını söyledi (ekrandan, 00:30:00). Bir e-ticaret sitemiz var ve farklı türde ürünler satıyoruz. Bir müşteri sipariş verdiğinde siparişte farklı türde ürünler olabilir ve çalışan bunları işleme koymalıdır. Sipariş işleme adımlarının belli bir sırası vardır, ama bu adımların iç mantığı ürün türüne göre çok farklılık gösterir.

Hoca anahtar noktanın “ürün türüne göre çok farklılık göstermesi” olduğunu vurguladı. Ortak temel adımlar arasında yalnız birkaç değişik adım olsaydı, bu iş ileride görülecek Template Method kalıbıyla daha kolay çözülürdü.

Çalışanın kullanacağı aracı şöyle düşünelim: elektronik bir ürün siparişi gelmişse çalışan elektronik dükkânı nesnesine ürünün tekil tanımlayıcısını (barkod ya da kendi tanımladığımız bir numara) vererek sipariş oluşturur. Kitap siparişinde kitap dükkânına kitabın tekil tanımlayıcısı (örneğin ISBN) verilir. Bir kitabın tanımlayıcısıyla elektronik dükkânında sipariş oluşturulmaya çalışılırsa “böyle bir ürün yok” yanıtı alınır; üstelik kitap siparişinin işlenmesi elektronikten çok farklıdır. Çalışan aslında komut yazmayacak, farklı sekmeleri olan bir grafik ön yüz kullanacaktır; ama arka planda yapılan iş, girilen dükkânın arayüzüne belirli bir tanımlayıcıya sahip ürünün siparişini vermektir.

Burada farklı türde ürünler ve onlarla çalışan farklı dükkânlar vardır; her dükkân tek bir tür ürünle çalışır. Ürün ailesinin farklı bileşenleri söz konusu değildir. Bu durumda Factory Method işe yarar. Hocanın deyişiyle Abstract Factory ile Factory Method arasında “dağlar kadar fark” vardır.

3.2 Çözüm şeması

Şema metne dökülünce (ekrandan, 00:40:00):

```
Order (soyut)  [Product]                    Store (soyut)  [Creator]
  + orderFromSupplier() {soyut}               # createOrder(type: String): Order {soyut}
  + pack()              {soyut}               + makeOrder()
  + prepare()           {soyut}
  + sendToCustomer()    {soyut}             Store - - «instantiate» - -> Order

Electronics : Order  [Concrete Product]     ElectronicsStore : Store  [Concrete Creator]
  + Electronics(String)                     ElectronicsStore - - «instantiate» - -> Electronics
  + orderFromSupplier(), pack(),
    prepare(), sendToCustomer()

Books : Order  [Concrete Product]           BookStore : Store  [Concrete Creator]
  + Books(String)                           BookStore - - «instantiate» - -> Books
  + orderFromSupplier(), pack(),
    prepare(), sendToCustomer()
```

Soyut `Store` sınıfında `createOrder` soyut metottur. Sipariş oluşturulurken yapılacak bazı temel işler vardır, ama siparişin gerçekten nasıl işleneceği ürüne göre çok değişir; hepsi tekil bir tanımlayıcıyla başlar ama alt adımları çok farklıdır. Ne tür ürünler satılıyorsa onların dükkân sınıfları `Store`'dan kalıtımla oluşturulur: `ElectronicsStore`, `BookStore`, `TextileStore` gibi. Örneğin giyim eklenecekse yalnız `Store`'dan türeyen bir `TextileStore` ve `Order`'dan türeyen bir tekstil ürünü sınıfı eklemek yeterlidir.

`Order` sınıfındaki metotların hepsi soyuttur; ortak bir nokta bulunamamıştır. Tedarikçiden sipariş vermek, paketlemek, hazırlamak ve müşteriye göndermek gibi işlemler vardır ama alt adımları çok farklıdır. Örneğin elektronik üründe garanti belgesinin işlenmesi ve tedarikçiden etkinleştirilmesi gerekebilir, kitapta böyle bir şeye gerek yoktur. Sipariş iptali ve iade koşulları da ürün türüne göre farklılaşabilir. Hoca bu problemde Template Method'un değil Factory Method'un en doğru seçim olduğunu yineledi.

3.3 Şemadaki bağımlılık okları üzerine

Şemadaki renkli sınıflar ve ilişki oklarının üstündeki «instantiate» stereotipleri, hocanın bir dönem Eclipse'te kaynak koddan otomatik UML şeması üreten eklentileri denerken oluşturduğu şekilden gelir. Normalde bu ilişki, sahiplik olmadan kesik çizgili bir okla (bağımlılık) çizilir. Bu ok şu anlama gelir: sınıf, okun gösterdiği türden bir nesneyi ya bir metoduna parametre olarak alıyor, ya bir metot içinde geçici olarak oluşturuyor ya da döndürüyor.

Hoca bu okların açıkça çizilmesini istedi: koddaki satır ilişkinin varlığını kanıtlasa bile ok çizilmelidir, çünkü oklar ilişkilere ve bağımlılıklara dikkat çeker. Bir tasarımı değerlendirirken az ok görüp bağlaşımı olduğundan düşük sanmak yanlış değerlendirmeye yol açar. Hoca eklentiyi sonradan bıraktığını da anlattı: ürettiği şemalar fazla kalabalıktı, okların birbirini kesmemesi için uzun uğraş gerekiyordu ve Eclipse'in yeni sürümlerinde desteği kesildi. Şemaları artık elle, istediği gibi çiziyor. Sınavlarda çizilen UML şemalarının kurallara ve güncel standarda uyması bir değerlendirme bileşeni olduğu için bu konuları fırsat buldukça tekrarladığını söyledi.

3.4 Kod

Kod, sınıf şemasının doğrudan iskelet koda aktarımıdır (ekrandan, 00:41:15):

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}
public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}
public class BookStore extends Store {
    protected Order createOrder(String uniqueID ) {
        Books newBookOrder = new Books(uniqueID);
        //Kitap siparişi için yapılacak işlemlerin tanımlanması
        return newBookOrder;
    }
}
```

`Store` kalıptaki Creator'a denk gelir. Hoca soyut metodu protected yapmayı tercih ettiğini, ama bunun bu aşamada şart olmadığını, kaynak kitapta da böyle geçtiği için değiştirmediğini

söyledi. Sipariş geldiğinde tekil tanımlayıcı parametre olarak alınır, ilgili sipariş oluşturulur ve sırayla hazırlama, tedarikçiden sipariş, paketleme ve müşteriye gönderme adımlarından geçilir. Her somut dükkân `createOrder` metodunu geçersiz kılmak zorundadır: elektronik dükkânı `Electronics` türünden, kitap dükkânı `Books` türünden bir nesne ilklendirir, gereken işlemleri yapar ve oluşturduğu sipariş nesnesini döndürür. Hoca Eclipse'teki projede de koda ek bir şey yazılmadığını gösterdi.

4. Builder Kalıbı

4.1 Amaç ve örnek sorun

Builder kalıbının amacı karmaşık bir nesnenin oluşturulmasını gösteriminden ayırmaktır. Böylece aynı oluşturma süreci farklı gösterimler ortaya çıkarabilir.

Örnek: Rich Text Format (RTF) metin okuyan bir program yazmak istiyoruz. Program okuduğu metni başka biçimlere de dönüştürebilmeli; hatta metni düzenleyebilecek bir grafik arayüz bileşeni de isteyebiliriz. Çok farklı hedef biçimler vardır ve hepsi hemen gerçeklenmeyecek, parça parça devreye alınacaktır. Bu yüzden yeni bir hedef dosya biçimi eklemek zor olmamalıdır. Zorluk yalnız o dosya biçiminin kendisiyle ilgili olmalı; hazır altyapıya bu biçimi destekleyecek sınıf ve metotları eklemek karmaşık olmamalıdır.

Önerilen çözüm: bir sınıf metni okumaktan, bir başka sınıf çevirmekten sorumlu olsun. Okuyucu, çeviriciye ilgili isteği göndersin (metodunu çağırsın). Çevirici nesne hem çevirme işleminden hem de çevrilen metni belirli bir biçimde simgelemekten sorumlu olsun; bunu daha fazla parçaya ayırmak mantıklı olmaz. Böyle yapılırsa yeni metin türleri için yeni çevirici alt sınıflar oluşturmak yeterli olur, başka bir yere dokunmak gerekmez.

4.2 Çözüm şeması ve eleştirileri

Şema metne dökülünce (ekrandan, 01:00:00):

```
RTFReader                        --builder-->   TextConverter (soyut)
  ParseRTF()                                       ConvertCharacter(char)   {soyut}
                                                   ConvertFontChange(Font)  {soyut}
                                                   ConvertParagraph()       {soyut}


ParseRTF() notu:
  while (t = get the next token) {
    switch t.Type {
      CHAR: builder->ConvertCharacter(t.Char)
      FONT: builder->ConvertFontChange(t.Font)
      PARA: builder->ConvertParagraph()
    }
  }

ASCIIConverter : TextConverter      TeXConverter : TextConverter       TextWidgetConverter : T
  ConvertCharacter(char)              ConvertCharacter(char)             ConvertCharacter(char
  GetASCIIText()                      ConvertFontChange(Font)            ConvertFontChange(For
                                      ConvertParagraph()                 ConvertParagraph()
                                      GetTeXText()                       GetTextWidget()
  - - -> ASCIIText                    - - -> TeXText                     - - -> TextWidget
```

Okuyucu bir RTF dosyasını ayrıştıracak bir metoda sahiptir ve ilgili isteği çeviriciye gönderir. Farklı hedef biçimler olacağı için çevirme metotlarını içeren soyut bir üst sınıf vardır; hangi biçime çevrilecekse o çevirme işlemini ilgili somut alt sınıf gerçekler. Gerçekte daha fazla işlem olabilir, ama şema kalabalıklaşmasın diye üç temel işlem gösterilmiştir. Hoca TeX'in Markdown'a benzer bir işaretlemeli metin olduğunu, `TextWidget`'ın ise metni düzenlemek için kullanılacak grafik arayüz bileşeni olduğunu söyledi.

Slayttaki eleştiriler şunlardır:

- Çok biçimlilik tam kullanılamamıştır; switch-case hâlâ durmaktadır. Okuyucu, ele aldığı her token'ın türünü denetlemek zorundadır: karakter bilgisi geldiyse karakter çevirme,

yazı tipi değişikliği geldiyse yazı tipi çevirme, paragraf geldiyse paragraf çevirme metodu çağrılır. Metnin her yeni bileşen türü için buraya bir case, üstteki soyut sınıfa bir soyut metot ve alt sınıflarda bu metodun gövdesi eklenmelidir.
- Alt sınıflar üst sınıfın bütün metotlarını gerçeklemelidir; bu yüzden alt sınıfta anlamı olmayan metotlar boş gövdeli kalmak zorundadır. Örneğin düz metinde (ASCII) yazı tipi bilgisi yoktur. Soyut sınıf kurallarına göre üst sınıfta soyut tanımlanan bir metot alt sınıfta ya somut olarak kodlanmalı ya da alt sınıf da soyut işaretlenmelidir; soyut işaretlemek işimize gelmediği için `ConvertFontChange` ve `ConvertParagraph` boş gövdeyle (`{ }`) yazılır.

Hoca kalıbın bir esneklik sağladığını, ama bu iki önemli zayıf noktası olduğunu söyledi.

4.3 Kalıp yapısı

Şema metne dökülünce (ekrandan, 01:01:00):

```
Director                --builder-->   Builder (soyut)
  Construct()                            BuildPart()  {soyut}
  notu: for all objects in structure {
          builder->BuildPart()
        }
                                       ConcreteBuilder : Builder  - - - ->  Product
                                         BuildPart()
                                         GetResult()
```

Slayttaki not: kalıbın yukarıdaki tasarımında switch-case görünmez, ama `ConcreteBuilder.buildPart` metodunda kendini gösterecektir. Çünkü `Director.construct` metodundaki "for all objects in structure" döngüsündeki nesneler bir biçimde `buildPart` metoduna parametre olarak aktarılmak zorunda kalınacak ve "parça şu ise şunu yap, bu ise bunu yap" gibi komutlar verilmek zorunda kalınacaktır. Hoca, bu ihtiyacın UML sınıf şemasında pek görünmediğini ama koda dökülünce ortaya çıktığını vurguladı.

Bileşenler:

- Builder: ürün nesnesinin parçalarının oluşturulması için bir arayüz sunar. Her farklı parça için farklı `BuildPart` metotları olabilir; bütün metotlar soyuttur.
- ConcreteBuilder: parçaları oluşturur ve birleştirir; işin nasıl yapılacağının bütün ayrıntısı burada kodlanır. Anlamı olmayan metotları boş gövdeyle kodlar. Ortaya çıkan ürünü döndürmek için bir getter metodu (`GetResult()`) vardır. ConcreteBuilder, ilişkili olduğu ürün türünün metotlarını istediği gibi kullanabilir.
- Director: oluşturma işini koordine eder ve yönlendirir. Ürün yapısındaki her parçanın hangi sırayla oluşturulacağını belirler ve builder'ın ilgili metodunu çağırır. Hocanın benzetmesiyle bir yönetici gibi "şunu, şunu yap" der; işi kendisi yapmaz.
- Product: ortaya çıkacak üründür (örneğin `TextConverter` için metin dosyası). Ürünün kendisini oluşturan parçalarla sahiplik ya da parça-bütün ilişkileri olabilir.

4.4 Bileşenlerin etkileşimi

Sıra şeması metne dökülünce (ekrandan, 01:04:30):

```
aClient                    aDirector                aConcreteBuilder
   |-- new ConcreteBuilder ------------------------------>|
   |-- new Director(aConcreteBuilder) ->|                  |
   |-- Construct() ------------------->|                  |
   |                                   |-- BuildPartA() ->|
   |                                   |-- BuildPartB() ->|
   |                                   |-- BuildPartC() ->|
   |-- GetResult() ------------------------------------------>|
```

İstemci Director'a "bana bir ürün lazım" der. Director ile Builder arasındaki ilişkiyi istemci kurar: istemci ne tür bir ürün istiyorsa onun ConcreteBuilder'ını oluşturup Director'a verir. Director ile soyut Builder arasında sahiplik ilişkisi olduğu için Director, Builder'ın herhangi bir somut alt sınıfıyla çalışabilir. Director ConcreteBuilder ile doğrudan ilişkili değildir; farklı türler her biri için ayrı somut alt sınıf üzerinden yapılır. Sonra istemci `Construct()` dediğinde

Director, kendisine verilmiş ConcreteBuilder'ı yöneterek parçaların gereken sırada ve biçimde oluşturulmasını sağlar. Ürün `Construct()` sonucunda döndürülebileceği gibi ayrı bir `GetResult()` metoduyla da alınabilir.

Hoca, önceki sınıf şemasıyla bu sıra şeması arasında birebir uyum olmadığını belirtti: sınıf şemasında tek bir `BuildPart` varken sıra şemasında `BuildPartA`, `BuildPartB`, `BuildPartC` vardır. Farklı parça türleri için farklı build metotları olabileceği unutulmamalıdır. Hoca bunu bir sonraki dönem için slayda not olarak düştü.

4.5 Fabrika kalıplarından farkı

Builder'da da ortaya bir ürün çıkar. Abstract Factory ve Factory Method'dan farkı, ürünün farklı alt parçalardan oluşmasıdır; farklı ürünlerin farklı alt parçaları da olabilir ve bunlar bir aile oluşturmaz. Bu yüzden Builder iki fabrika kalıbından çok farklı bir kalıptır.

4.6 Uygulanabileceği anlar ve sonuçlar

Slayttaki maddeler (ekrandan, 01:10:00):

- Kalıbın uygulanabileceği anlar:
  - Karmaşık bir nesneyi oluşturma algoritmasının, nesneyi oluşturan parçalardan bağımsız olması ve bu parçaların nasıl birleştirileceğinden bağımsız olması gerektiği anlarda.
  - Bir nesneyi oluşturma sürecinin bu nesnenin farklı gösterimlerinin olmasına izin vermesi gerektiği anlarda (örneğin metin içeriğini farklı gösterimlere çevirmek).
- Sonuçlar:
  - Ürünün iç yapısının en az yan etkiyle değiştirilebilmesi mümkün olur (yeni bir ConcreteBuilder sınıfı yazarak).
  - Önceden görülen "factory" kalıplarıyla karşılaştırıldığında, ürün oluşturma süreci üzerinde daha fazla denetim sahibi olunur (Director örneği, Builder örneğine hangi işlemi hangi sırada yapması gerektiği emrini verir).

Birinci sonucu hoca bir örnekle açıkladı: RTF örneğine PDF biçimi eklenmek istensin. Yalnız `TextConverter`'ın alt sınıfı olarak bir `PDFConverter` sınıfı oluşturulur ve PDF çevirmeyle ilgili bütün işlemler bu sınıfa kodlanır. Özgün metin biçimine (RTF) dokunulmaz, ayrıştırıcıdaki switch-case'e de, kardeş sınıflara da dokunulmaz. Kardeş sınıflara dokunmak gerekiyorsa tasarım hatalı demektir.

İkinci sonuç, bir ürün oluşturma problemiyle karşılaşıldığında hangi kalıbın seçileceğine karar verirken düşünülecek ölçütlerden biridir. Director, Builder'ı her seferinde "şu karakteri çevir, yazı tipini çevir, paragrafı çevir" diye adım adım yönetir. Abstract Factory ya da Factory Method'da ise istemci "şu aileden şu ürünü istiyorum" der ama ürünün nasıl oluşturulacağına müdahale etmez, edemez; o kalıplar bunu desteklemez. Builder ise oluşturma sürecine yakın müdahaleye ve denetime izin verir.

5. Builder Örneği: Araç Adlarını Çözümleyen Modül

5.1 Problem

Bu örneği hoca kendisi oluşturmuştur. Bir oto galerisi yazılımı ya da belirli bir markanın yetkili bayisi için bir ürün geliştiriyoruz; ama ürünü tek bir markaya değil, farklı markaların bayilerine de (örneğin Renault, Fiat) satabilmek istiyoruz. Farklı marka ve modellerin adlandırmaları çok farklıdır. Örneğin Renault Megane hatchback ya da sedan olabilir, Renault Captur ise her zaman SUV'dir. Donanım paketlerinin adları da markadan markaya çok farklıdır. Yazılımda, aracın adından hangi özelliklere sahip olduğunun bulunabileceği bir modül olacaktır ve örnekte bu modül modellenmektedir.

Söz konusu özellikler: kasa şekli, yakıt türü, motor hacmi, motor gücü ve vites. Vites bilgisi vitesin kaç ileri olduğunu ve otomatik olup olmadığını; motor bilgisi motor hacmini ve beygir gücünü ifade eder. Hoca motor hacminin tam sayı olarak tutulacağını söyledi: "1.4 motor" yerine 1400 yazılır. Kasa türleri sedan, hatchback, station wagon ve SUV olabilir; pick-up satan

bir markaya da hitap edilecekse enum'a yeni bir seçenek eklenir. Yakıt türü benzin, dizel, LPG, elektrik ya da hibrit olabilir.

Aracın adındaki her bileşen (birkaç harf ya da kelime) ayrı bir anlam ifade ettiğine göre, farklı parçalardan oluşmuş karmaşık bir ürün söz konusudur. Bu tür bir problemi çözmek için Builder kalıbı uygulanır.

5.2 Tasarım

Şema metne dökülünce (ekrandan, 01:20:00):

```
paket product:
  Car
    - brand, model : String

paket example:
  CarDirector                                «abstract» CarBuilder
    - builder : CarBuilder      1 ---> 1       - product : Car
    + CarDirector( CarBuilder )                + CarBuilder( carModel : String )
    + construct( elements[] : String )         # getProduct( ) : Car
    + report( )                                + getResult( ) : Car
                                               + buildChassis( ) {abstract}
                                               + buildEngine( token : String ) {abstract}
                                               + buildTransmission( token : String ) {abstract}
                                              CarBuilder ---> Car

  RenaultBuilder : CarBuilder                  HondaBuilder : CarBuilder
    + RenaultBuilder( carModel : String )        + HondaBuilder( carModel : String )
    + buildChassis( )                            + buildChassis( )
    + buildEngine( token : String )              + buildEngine( token : String )
    + buildTransmission( token : String )        + buildTransmission( token : String )
```

Kalıp gereği soyut bir `CarBuilder` sınıfı oluşturulur. Ortaya çıkacak ürün bir üye alan olarak saklanır: tek bir build metodu olsaydı ürünü doğrudan döndürmek yeterdi, ama farklı parçalar yavaş yavaş eklendiği için ürünün üye alan olarak tutulması gerekir. `CarBuilder`'ın kurucusu, aracın bütün özelliklerini içeren uzun model adını parametre olarak alır. Sonuç `getResult()` ile alınır; somut alt sınıfların ürüne erişmesi gerektiği için `getProduct()` metodu protected yapılmıştır. Geri kalan sınıf ve metotlar kalıbınkiyle aynıdır.

Yazılım bir Renault bayisine satılacaksa `CarDirector` bir `RenaultBuilder` nesnesiyle ilişkilendirilerek yapılandırılır; bir Honda bayisine satılacaksa `CarDirector`'a bir `HondaBuilder` verilir. Yazılım kendi sunucumuzda hizmet olarak sunuluyorsa da ilgili ilişkilendirme orada yapılır.

5.3 Kod üzerinden inceleme

`product` paketindeki sınıflarda dikkat çeken tek nokta, primitif olmayan, sınıf olan bir Java enum'unun kullanılmasıdır. Kasa sınıfından olması gereken nesneler yalnız sedan, hatchback, station wagon ve SUV olacaksa bu kadarıyla bırakmak primitif bir enum olurdu. Ama bunlar aslında bir sınıfın nesneleridir ve üye alanları ile metotları olabilir. Bu gösterimde her değerin yanında parantez içinde verilen parametre, aslında enum kurucusuna verilen parametredir; yani her değer bir kurucu çağrısıdır.

`CarDirector.construct` metoduna hoca, model adının parçalarını önceden ayrılmış hâlde verdi; ayrıca bir ayrıştırma (tokenize) yapmak istemediğini söyledi. Her öğe bir sekmeyle ayrılmış iki alt parçadan oluşur: parça türü (`Engine`, `Gear`) ve değeri. Director öğenin türüne göre `buildEngine` ya da `buildTransmission` metodunu çağırır; motor bilgisi yakıt türünü de içerir, vites bilgisi hem otomatik ya da düz olduğunu hem de kaç ileri olduğunu birlikte içerir. Sonucu almak için `report()` metodu eklenmiştir; builder'dan sonucu alıp ekrana yazar.

Hoca, kasa tipini model adına bağladığını anlattı. Örneğin `RenaultBuilder.buildChassis()` model adını büyük harfe çevirip (ignore case ile aramaktansa kolaya kaçtığını söyledi) belirli kelimeleri arar: Captur her zaman SUV'dir; kurala uymayan modeller hatchback sayılır. Renault ileride bu dallanmadan farklı değerlendirilmesi gereken adlarla bir model çıkarırsa yalnız

bu kısım değişir; başka markaların ve modellerin kısımlarına hiç dokunulmaz. Değişiklik olması gerektiği gibi tek bir noktada yapılır.

`HondaBuilder.buildChassis()` de benzer çalışır: adında "back" geçiyorsa (fastback gibi) kasa hatchback'tir; tek bir SUV modeli için ayrı bir kural vardır; geri kalanlar sedandır.

5.4 İki builder'ın karşılaştırılması

Hoca iki somut builder'ı Eclipse'in karşılaştırma aracıyla yan yana gösterdi (ekrandan, 01:30:00). `buildEngine` metotlarının görünen kısmı:

```
// HondaBuilder
public void buildEngine(String token) {
    //Min. values for a Honda engine
    if( getProduct().getEngine() == null )
        getProduct().setEngine( new Engine(FuelType. ... ) );   // satırın devamı ekranda görü
    token = token.toUpperCase();
    if( token.contains("ECO") )
        getProduct().getEngine().setFuel(FuelType.LP ... );     // satırın devamı ekranda görü
    else if( token.contains("PS") ) {
        int end = token.indexOf("PS");
        token = token.substring(0, end);
        getProduct().getEngine().setPower( Integer.p ... );     // satırın devamı ekranda görü
    }
    ...
}

// RenaultBuilder
public void buildEngine(String token) {
    ...
    token = token.toUpperCase();
    if( token.contains("DCI") )
        getProduct().getEngine().setFuel(FuelType. ... );       // satırın devamı ekranda görü
    else if( token.contains("BG") ) {
        getProduct().getEngine().setPower( Integer ... );       // satırın devamı ekranda görü
    }
    else if( token.contains(".") ) {
        getProduct().getEngine().setVolume( (int)( ... );       // satırın devamı ekranda görü
    ...
}
```

Hocanın anlatımına göre Honda'da motor bilgisinde "ECO" geçiyorsa motor LPG'lidir; "PS" geçiyorsa "PS"den önceki sayı beygir gücüdür. Renault'da gücü "BG" belirtir. Vites için Honda'da "AT" otomatik vites demektir; Renault'da "CVT" ya da "EDC" geçiyorsa vites otomatiktir. Hoca bu kuralların bir kısmını örnek için uydurduğunu, kodun yüzde yüz farklı olmadığını, kalıbı yeterli düzeyde örneklemek için kritik birkaç noktada farklılaştırdığını söyledi.

Hoca ayrıca string'lerde switch-case yerine if-else zinciri kurduğunu belirtti; bu if-else zinciri kalıbın standart örneğindeki switch-case kısmına denk gelir: hangi parça gelmişse onun oluşturulması için ilgili işlemler yapılır.

> **Ders sonrası not:** Hoca string'in switch-case'e verilemeyeceğini söyledi. Java 7'den beri `switch` ifadesi `String` değerlerle de kullanılabilir. Burada ise koşullar tam eşitlik değil `contains` araması olduğu için if-else zinciri yine de gereklidir.

5.5 Çalıştırma

`CarDirector.main` (ekrandan, 01:28:10):

```
public static void main(String[] args) {
    CarDirector director = new CarDirector( new HondaBuilder("Civic Sedan") );
    String elements[] = {"Engine\t125PS","Engine\t1.6L","Gear\t6AT"};
```

```
    director.construct(elements);
    director.report();
}

public void report( ) {
    System.out.println(builder.getResult());
}
```

Çözümleme adım adım şöyledir (derste çalıştırıldı, 01:32:43):

- Model adı “Civic Sedan”: Honda kuralına göre adında “back” ya da SUV modelinin adı geçmediği için kasa sedandır.
- “Engine125PS”: “PS”den önceki kısım 125, yani motor gücü 125 beygirdir.
- “Engine1.6L”: nokta içeren değer motor hacmidir; 1.6 litre tam sayı olarak 1600 cc tutulur.
- “Gear6AT”: “AT” otomatik vites demektir, önündeki 6 ileri sayısıdır.
- Çıktı: sedan kasa, benzinli, 1600 cc, 125 beygir, 6 ileri otomatik vites.

Hoca, burada önemli olanın bilgilerin düzgün ayrıştırılması değil, sonucu oluşturan parçaların kim tarafından, nerede ve nasıl birleştirildiği olduğunu vurguladı.

6. Prototype Kalıbı

6.1 Amaç ve örnek

Prototype kalıbının amacı, oluşturulacak nesnelerin türlerini belirlemek için bir prototip nesne kullanmak ve yeni nesneleri bu prototipi kopyalayarak oluşturmaktır (ekrandan, 01:50:00). Hoca, tamamen bu mantıkla çalışan diller de olduğunu, ama akademik kaldıkları ve sahada pek kullanılmadıkları için derse girmeyeceğini söyledi.

Örnek: müzikle ilgileniyoruz ve nota biliyoruz. Elimizde hazır bir grafik uygulama çerçevesi var ve kodlamayı biliyoruz; bu ilgi alanına hitap eden bir müzik besteleme programı yazmak istiyoruz. Beste yapmak ya da birinin söylediği notaları kâğıda geçirmek için bir porte çizmek gerekir. Portenin beş çizgisi grafik programındaki çizgi nesneleriyle çizilir; sol anahtarı biraz daha zordur. Her nota için bir elips çizilir; elipsin içi dolu ya da boş olabilir, notanın kuyruğu için dik bir çizgi eklenir. Aynı şekil hangi aralığa ya da çizginin üstüne çizildiğine ve içinin dolu ya da boş olmasına göre farklı notaları gösterir. Kuyruklarından birleşik notalar da vardır. Az sayıda temel şekil olduğu için grafik bileşenleriyle böyle bir program yazılabilir.

Slayttaki örnek ve sorun:

- Çerçeve programının soyut bir `Graphic` sınıfı, çeşitli kontrol düğmelerinin yer alabileceği paletleri simgeleyen soyut bir `Tool` sınıfı, bundan kalıtımla türetilmiş ve yeni grafik şekillerini çizim alanına eklemeye yarayan `GraphicTool` sınıfı vardır.
- Notaları eklemek için `GraphicTool` sınıfı kullanılabilir.
- Sorun: çerçeve programının çeşitli nota sınıfları hakkında önceden bilgisi olamaz; çerçeveyi yazan kişinin aklına onun bir müzik besteleme aracında kullanılacağı bile gelmeyebilir. `GraphicTool` sınıfı bilmediği sınıflardan çizim nesnelerini nasıl oluşturur?

Hoca palet derken Paint.NET’teki gibi bir araç kutusunu kastettiğini gösterdi: her bileşeni tıklandığında farklı eylemlerin kodlanabildiği bir yapı.

6.2 Çözüm şeması

Şema metne dökülünce (ekrandan, 02:00:00). Hoca şemayı kesik bir çizgiyle böldü: sol ve üst taraf çerçeveye, sağ alt taraf bizim uygulamamıza aittir.

```
Çerçeve:
  Tool (soyut)                          Graphic (soyut)
    Manipulate()  {soyut}                 Draw(Position)  {soyut}
                                          Clone()         {soyut}
  RotateTool : Tool
    Manipulate()
  GraphicTool : Tool  --prototype-->  Graphic
    Manipulate()
```

```
    notu: p = prototype->Clone()
          while (user drags mouse) {
              p->Draw(new position)
          }
          insert p into drawing

Uygulama:
  Staff : Graphic                        MusicalNote : Graphic (soyut)
    Draw(Position)                         WholeNote : MusicalNote      HalfNote : MusicalNote
    Clone()                                  Draw(Position)               Draw(Position)
                                             Clone()                      Clone()
                                             notu: return copy of self    notu: return copy of s
```

Dörtlü Çete'nin örneğinde çerçeve Prototype kalıbının altyapısını destekler ve uygulama bu altyapıyı kullanarak notaları oluşturur. `Tool`'un temel işlemi bir şeyi biçimlendirmektir; bu yüzden soyut `Manipulate()` metodu vardır. Somut `RotateTool` ve `GraphicTool` sınıfları `Tool`'dan türer. `RotateTool` da işe yarar, çünkü ters dönmüş notalar vardır; `GraphicTool` ise notaları ve porteyi çizmek için kullanılır.

6.3 RotateTool ile GraphicTool arasındaki fark

`GraphicTool` ile yönettiği `Graphic` arasında bir parça-bütün ilişkisi vardır; bir `GraphicTool` birden fazla grafik nesnesini yönetebilir. Hoca, `RotateTool` da grafikleri döndüreceği hâlde neden onunla `Graphic` arasında böyle bir ilişki olmadığını sordu ve kendisi yanıtladı: bir şekli döndürmek için onun metin mi, kare mi, daire mi olduğunu bilmek gerekmez. Bilgisayar grafiği açısından döndürme, ilgili grafiği bir döndürme matrisiyle çarpmaktan ibarettir. `GraphicTool`'un yapabileceği değişiklikler ise şeklin kendisine daha bağımlıdır: kare, üçgen ya da daire içi dolu ya da boş çizilebilir; metin italik yapılabilir; Bezier eğrisinin denetim noktaları değiştirilebilir. Bu ilişki bütün kardeş alt sınıflarda olsaydı onu soyut `Tool` üst sınıfına taşımak en doğrusu olurdu; ama öyle değildir.

`GraphicTool`'daki üye alanın adı, kalıbın kullanıldığını göstermek için `prototype` seçilmiştir. Bu alan bir dizi ya da generic bir bağlı liste gibi bir veri yapısı olabilir; yani çoklu sahiplik (parça-bütün) ilişkisi vardır.

6.4 `Manipulate()` metodunun çalışması

`GraphicTool.Manipulate()` soyut değil somut bir metottur. Hoca, metodun adının pek mantıklı olmadığını, metodun aslında grafik nesnesini tutup resmin içinde sürüklerken yapılan işi, yani sürükle-bırak işlemini anlattığını söyledi. Bir çizim aracında sağdaki paletten bir şekil tıklanıp çizim alanına bırakıldığında olan budur. Sürükleme kesintisiz görünür ama aslında küçük adımlarla yapılır: her küçük adımda eski konumdaki nesne silinir ve yeni konumda yeni bir nesne çizilir.

Yeni nesneyi sıfırdan oluşturmaktansa önceki nesnenin bir kopyasını (klonunu) oluşturmak, en azından kod açısından daha hızlı ve pratik bir çözümdür. Bu yüzden `Clone()` metodu mutlaka olacaktır. Porte örneğinde de yeni bir sayfaya ya da yeni satıra porte çizileceğinde üstteki porte klonlanır; nota sürüklenirken de klon prototip eski yerinden silinip yeni yerine taşınır. Notalar (tam nota, yarım nota vb.) kendi aralarında kardeş sınıflar olarak özelleşir ve `Clone()` metodu bütün somut sınıflarda gerçeklenmelidir.

Hoca güncel UML standardı açısından bir eksiği de belirtti: `Clone()` nesnenin kendisinin bir kopyasını döndürdüğü için dönüş türü de gösterilmelidir; yani şemada `Clone() : Graphic` biçiminde yazılmalıdır.

6.5 Neden Prototype? Fabrika kalıplarıyla karşılaştırma

Bu yapı başka kalıplarla da kurulabilir. Abstract Factory ile çözülürse yarım nota ailesi, tam nota ailesi, ters nota ailesi gibi aileler olur ve do'dan si'ye kadar notalar o ailenin üyeleri olur. Factory Method ile çözülürse tam nota, yarım nota gibi her nota türü için ayrı fabrika sınıfları

gerekir. Abstract Factory'de aileler ve üyeleri derken daha fazla sınıf tasarlamak gerekir. Prototype ile ise yalnız nota türü kadar, daha az sayıda sınıfla problem çözülür; Prototype bütün ihtiyaçları daha basit karşılar. Dörtlü Çete de bu örnekte Prototype'ı seçmiştir.

Hoca önce bellek optimizasyonunu da bir neden olarak saydı, ama hemen düzeltti: bellek optimizasyonu daha çok Flyweight kalıbıyla sağlanır. Öğrencileri bu problemi ileride görecekleri başka kalıplarla da çözmeyi denemeye ve yararlı buldukları çözümleri paylaşmaya davet etti.

6.6 Kalıp yapısı

Şema metne dökülünce (ekrandan, 02:10:00):

```
Client  --prototype-->  Prototype (soyut)
  Operation()             Clone()  {soyut}
  notu: p = prototype->Clone()

ConcretePrototype1 : Prototype          ConcretePrototype2 : Prototype
  Clone()  notu: return copy of self      Clone()  notu: return copy of self
```

- Prototype: istemci nesnelerin kendisini kopyalaması ve kullanması için bir arayüz sunar. Soyut `Clone()` metodu, gövdesinin somut alt sınıflarda gerçeklenmesi gerektiğini hatırlatır. Şemada gösterilmese de prototip nesneleriyle yapılabilecek işlemler de (ortak ise somut, değilse soyut metotlar olarak) bu üst sınıfta yer almalıdır; yalnız prototip oluşturmak bir işe yaramaz. Bir metot yalnız belirli bir alt sınıfa özelse ona ulaşmak için tür dönüşümü (type casting) gerekir; bu pek güzel bir şey değildir.
- ConcretePrototype: kullanılacak nesnelerin sınıflarıdır.
- Client: bir prototipten kendisinin kopyasını isteyerek yeni nesne oluşturur. `Operation()` içinde başka işlemler de yapar, ama bunların içinde mutlaka bir prototip nesnenin kopyasını oluşturup onun üzerinde çalışır.

6.7 Kalıbın uygulanabileceği anlar

Slayttaki maddeler (ekrandan, 02:20:00):

- Bir sistemin, kendisine ait ürünlerin nasıl oluşturulduğu, birleştirildiği ve sunulduğundan bağımsız olmasının istendiği durumlar,
- ve şu durumlardan biri söz konusu ise (Factory Method kalıbından farklılaşmayı sağlıyor):
  - Oluşturulacak sınıflar çalışma anında belirlenecekse,
  - Ürünlerin sınıf hiyerarşisi ile paralel bir factory hiyerarşisi kurmak istenmiyorsa,
  - Ürün sınıfının örneklerinin durum uzayı sınırlı ise.
    - Bu durumda sınıfın iyi tasarlanıp tasarlanmadığı ayrıca tartışılır!
- Bu kalıp C++ gibi sınıfların doğrudan işlenebilecek varlıklar olmadığı dillerde daha çok yarar sağlar.
  - SmallTalk gibi geç ilişkilendirme (late binding) ve Java gibi yansıtma (reflection) yetenekleri olan dillerde bu kalıp yerine dilin yetenekleri kullanılabilir.

Hoca ilk maddeyi şöyle açıkladı: oluşturma, birleştirme ve sunma ayrıntılarının hepsi somut prototip sınıflarının `clone()` metotlarında kodlanır; istemci bunlar hakkında hiçbir şey bilmeden kopyasını oluşturur. Bu üç durumdan biri yoksa sorun Factory Method ile de çözülebilir ve o tercih edilir.

Çalışma anında belirleme konusunda: bir alt sınıfın `clone()` metodu kendisiyle aynı türden bir nesne döndürebileceği gibi, varsa kendi alt sınıfı ya da bir kardeş sınıf türünden bir nesne de oluşturup döndürebilir. Kalıtımda kardeş sınıfların birbirinden haberdar olması normalde istenmez; ama problemin çözümünü kolaylaştırıyorsa yapılabilir ve Prototype buna izin verir. Factory Method'da bunu yapmak, zaten biraz daha karmaşık olan kalıbı daha da karmaşık hâle getirir.

Paralel hiyerarşi konusunda: ürünler arasında bir mantıksal hiyerarşi varsa ve bu bir sınıf kalıtım hiyerarşisi olarak görünüyorsa, Factory Method kullanıldığında fabrika sınıfları arasında da aynı paralel hiyerarşiyi kurmak gerekir. Prototype'ın Factory Method yerine seçilmesinin bir nedeni de budur.

Durum uzayı konusunda hoca, az sayıda prototip olmasının tercih edilen bir durum olduğunu, çünkü prototip kategorileri arttıkça hepsinin izini sürmenin zorlaştığını söyledi. Ancak "bu durumda sınıfın iyi tasarlanıp tasarlanmadığı ayrıca tartışılır" alt maddesinin gerekçesini derste çıkaramadığını, daha sonra düşüneceğini belirtip slayda not düştü.

> **Ek bilgi (derste anlatılmadı):** Dörtlü Çete'nin kitabında bu madde, bir sınıfın örneklerinin yalnız birkaç farklı durum birleşiminden birine sahip olabildiği durumu anlatır. Bu durumda her seferinde sınıfı uygun durumla elle oluşturmak yerine o sayıda prototip kurup bunları klonlamak daha kullanışlı olabilir. Slayttaki uyarı büyük olasılıkla şunu kastediyor: örnekler yalnız birkaç sabit durumda bulunuyorsa, bu durumların ayrı alt sınıflar ya da bir enum gibi başka bir yapıyla modellenmesinin daha uygun olup olmadığı sorgulanmalıdır.

6.8 Geç ilişkilendirme ve yansıtma

Daha geleneksel, genel geçer nesneye yönelimde sınıflar doğrudan işlenip değiştirilebilen varlıklar değildir: sınıfın kodu bir kez yazıldıysa üye alanları ve metotları değişmez; değişmesi gerekiyorsa yeniden yazılır. SmallTalk ise nesneye yönelimin temellerinin atıldığı ilginç bir dildir ve geç ilişkilendirme (late binding) sunar. Hoca bunu Visual Basic'te türü belirtilmeden tanımlanan ve türü ilk atamada belli olan değişkenlere benzetti; aynı şeyi nesneler için, sınıfı belirtilmeden tanımlanan bir nesne işaretçisi olarak düşünmek gerekir. Böyle bir dilde Prototype kalıbına hiç gerek yoktur. C++ gibi kuvvetli tip denetimli dillerde ise bu iş yapılamaz.

Java'da hocanın çok kullandığı Reflection API vardır. Herhangi bir nesnenin sınıfını bilmek gerekmez; `Object` türünde bir işaretçiyle alınan nesnenin hangi sınıftan olduğu, hangi üye alanları ve metotları olduğu programla sorgulanabilir ve kullanılabilir. Bu, Java'nın standart bir özelliğidir; C#'ta da benzer bir yetenek vardır. Apache'nin BCEL (Bytecode Engineering Library) kütüphanesi bunu bir adım öteye götürür: bir nesnenin sınıfının bytecode'unu çalışma anında değiştirmeyi sağlar. Hocanın örneğinde, yalnız `ad` üyesi olan `İnsan` sınıfının kaynak koduna dokunmadan, çalışma anında belirli bir insan nesnesine bir de `soyad` üyesi eklenebilir.

Soru: Java ve Dart derlenen diller; yansıtma ile çalışma anında bir nesnenin yapısını sorgulamak ve değiştirmek nasıl mümkün oluyor? **Cevap:** Hoca konuda ayrıntılı bilgisi olmadığını, ama bunun Java'da bir çalışma ortamı (Java Runtime Environment, sanal makine) bulunması sayesinde mümkün olduğunu söyledi. Dart'ta da bir çalışma ortamı vardır. Java'da bir sınıfın kodu belleğe gerektiğinde (just in time) sınıf yükleyici (class loader) bileşeni tarafından yüklenir ve bu bileşen sanal makineyle birlikte değiştirilebilir. Böyle bir çalışma ortamı olmayan dillerde bu işler ya mümkün olmaz ya da çok daha zordur.

> **Ders sonrası not:** Hoca Java'nın "yorumlanan bir dil" olduğunu söyledi. Daha doğrusu, Java kaynak kodu derleyiciyle bytecode'a derlenir; bytecode da Java sanal makinesinde (JVM) yorumlanarak ve sık çalışan kısımları JIT derleyiciyle makine koduna çevrilerek çalıştırılır. Yansıtmayı mümkün kılan, sınıf bilgisinin (metadata) bytecode'da ve çalışma ortamında korunmasıdır.

6.9 Sığ kopyalama ve derin kopyalama

Prototype'ta önemli bir konu, nesnelerin kendilerini sığ (shallow copy) mı yoksa derin (deep copy) kopyalama kipinde mi klonlayacağıdır. Hoca ilişki zinciri örnekleri verdi: bir insan ve sahip olduğu araba, bir öğrenci ve aldığı dersler, bir doktor ve baktığı hastalar, bir hasta ve açık tedavi işlemleri ile bunları reçete eden doktor.

- Sığ kopyalama: yalnız nesnenin kendisi kopyalanır; primitif olmayan üye alanlarının gösterdiği nesneler kopyalanmaz. Bu bir bilgi kopmasına yol açabilir; sakıncalı olduğu da olmadığı da durumlar vardır. Klonlamadan sonra bütün ilişkiler zaten yeniden kurulacaksa sığ kopyalamanın sakıncası yoktur; daha hızlı ve performanslıdır.
- Derin kopyalama: ilişkileri yeniden kurmaktan kaçınmak için gerekir. Nesne klonlanırken üye alanı olan diğer nesneler de çoğaltılır, onların da primitif olmayan üye alanları kopyalanır; zincirleme bir kopyalama yapılır.

Hoca bunu Java'daki nesne serileştirmeye (serialization) benzetti: bir nesneyi bytecode'a çevirip disk dosyasında ya da bayt dizisinde saklamak ya da başka bir Java sanal makinesine göndermek gibi işlerde de aynı sorun vardır. Bir nesne ilişkili olduğu diğer nesnelerle aynı

dosyaya ya da bağlantıya serileştirilmezse ilişki kopar. Örneğin bir hasta nesnesi ile onu tedavi eden doktor nesnesi ayrı dosyalara serileştirilirse, öbür bilgisayarda hasta nesnelerinin nesne tanımlayıcıları aynı kalırken doktor nesnelerininki farklı olur. Hoca burada, `toString` geçersiz kılınmadığında nesne yazdırılınca çıkan onaltılık sayıdan söz etti; ona bakarak iki işaretçinin aynı nesneyi gösterip göstermediği anlaşılabilir.

> **Ders sonrası not:** Hoca bu değeri "Universal Unique ID" olarak andı. Java'da `toString` geçersiz kılınmadığında yazdırılan değer `SınıfAdı@` ve ardından nesnenin `hashCode()` değerinin onaltılık gösterimidir; varsayılan `hashCode()` kimlik tabanlıdır ama evrensel olarak tekil bir kimlik (UUID) değildir ve farklı nesnelerde nadiren de olsa aynı çıkabilir. İki işaretçinin aynı nesneyi gösterip göstermediği kesin olarak `==` ile anlaşılır.

"Her zaman derin kopyalama yapalım, biraz uğraştırsın" denebilir; ama ilişkilerde bir çevrim varsa sonsuz döngü gibi garip durumlar ortaya çıkabilir. Buna mutlaka dikkat etmek gerekir.

6.10 Kalıbın zayıf bir yanı

Klonlar çok farklı türlerden olacaksa ve farklı türlerin farklı üyeleri gerekiyorsa kalıbın zayıf bir yanı ortaya çıkar. Hoca nota örneğinden yola çıktı: tekli notanın yanında kuyruğundan birleştirilmiş çiftli nota oluşturmak istenirse nota sınıfına "bağlı kardeş ekle" gibi bir metot gerekebilir. Bazı alt sınıflarda anlamı olan, bazılarında olmayan metotlar söz konusuysa bunların hepsini üstteki soyut prototip sınıfında tanımlamak gerekir. Bu, Builder'daki `ASCIIConverter`'ın paragraf metodunu boş gövdeyle yazma zorunluluğuyla aynı sakıncadır. Ancak farklı alt türlere ihtiyaç yoksa prototip sınıfı tek bir somut sınıf olarak da yapılabilir; o zaman bu sorunların çoğu ortadan kalkar.

7. Prototype Örneği: Oyun İçin Bölüm Düzenleyicisi

7.1 Problem

Örnek Head First Design Patterns kitabındandır. Bir bilgisayar oyunu için bölüm düzenleyicisi (level editor) hazırlanıyor; amaç oyun haritasına farklı türden canavarları çok kolay ekleyebilmektir. Bir canavar türü oluşturmak kolay değildir, çünkü canavarın birçok durum bilgisi vardır: HP (can puanı, canavarın sağlığı), DEX (el çabukluğu, örneğin saniyede kaç kez ateş edebildiği), AGL (çeviklik, örneğin isabetlerden kaçabilmesi), LUCK (şans), CHR (karizma) gibi. Bunların hepsi `int` olarak tutulur. Canavarın bir de tipi vardır: Ork, Dark Elf gibi.

Prototype burada kolay bir çözüm sunar. Temelde hepsi canavardır ama bazı özellikleri farklılaşır. Temel bir canavar prototipi klonlanıp bazı durum bilgileri değiştirilerek oyuna yeni bir canavar türü eklenebilir.

7.2 Tasarım ve kod

Soyut `Monster` sınıfı prototiptir ve bütün durum bilgilerini üye alan olarak tutar. Ondan somut bir `GenericMonster` sınıfı türetilir; bu somut prototiptir. Oluşturulan canavar türlerini kaybetmemek ve yeniden kullanmak için bir `MonsterRegistry` sınıfı vardır; her canavar türünün `type` değeri onun anahtarıdır. Bölüm sonu canavarı mı gerekiyor, o türle aranır; minyon mu gerekiyor, o anahtarla aranır ve bulunan canavar döndürülür. Bir canavarı değiştirip yeni bir tür oluşturmak için onu klonlayıp ilgili üyelerini değiştirir ve yeni bir adla kaydederiz.

`Monster` sınıfı (ekrandan, 02:36:10):

```
package dp.prototype.example;
public abstract class Monster {
    protected int HP, DEX, AGL, LUCK, CHR;
    protected String type;

    public abstract Monster clone( );

    protected Monster(int hp, int dex, int agl, int luck, int chr) {
        HP = hp; DEX = dex; AGL = agl; LUCK = luck; CHR = chr;
```

```
    }
    public String getType() { return type; }
    public int getHP() { return HP; }
    protected void setHP(int hP) { HP = hP; }
    public int getDEX() { return DEX; }
    protected void setDEX(int dEX) { DEX = dEX; }
    public int getAGL() { return AGL; }
    protected void setAGL(int aGL) { AGL = aGL; }
    public int getLUCK() { return LUCK; }
    protected void setLUCK(int lUCK) { LUCK = lUCK; }
    public int getCHR() { return CHR; }
    protected void setCHR(int cHR) { CHR = cHR; }
    protected void setType(String type) { this.type = type; }
}
```

`clone()` soyut metottur, çünkü belki bir alt sınıfta derin, başka bir alt sınıfta sığ kopyalama yapılacaktır. Kurucu bütün bilgileri alır. Getter'lar public'tir. Slayttaki kutuda şu yazar: "protected setter'lar ile erişimin tek yolu Register olur, tercih ederim." Hoca, klonlanan nesnenin niteliklerini değiştiren setter'ları public yapmadığını açıkladı: alt sınıflar bunları rahatça değiştirebilsin, ama başka bir istemci değiştiremesin. Kaynak kitapta da protected kullanıldığını söyledi. Java'da `Monster` ile `MonsterRegistry` aynı pakette olduğu için Registry bu metotlara erişebilir; paket dışındaki bir sınıf ise oyunu kendi amacına göre değiştiremez.

`GenericMonster`, `Monster`'dan türeyen somut sınıftır: kurucusu üst sınıfın kurucusunu çağırır ve tipini "Generic Monster" olarak atar. `clone()` metodu kendisinin yeni bir kopyasını döndürür: kendisiyle aynı durum bilgisine sahip ama başka bir nesne. `MonsterRegistry`, anahtarı tip adı (`String`), değeri `Monster` nesnesi olan generic bir `HashMap` tutar. Kurucusu `HashMap`'i ilklendirir; `findMonster` metodu `HashMap`'in `get` metodunu çağırıp o anahtarlı canavar varsa döndürür; kaydetmek için `put` metodu kullanılır.

Ana program (ekrandan, 02:40:00):

```
public class MainApp {
    public static void main(String[] args) {
        MonsterRegistry reg = new MonsterRegistry();
        reg.registerMonster( new GenericMonster(10, 10, 10, 10, 10) );
        Monster mySpecialMonster =
                reg.findMonster("Generic Monster").clone();
        mySpecialMonster.setHP(100);
        mySpecialMonster.setType("Bolum sonu canavarı");
        reg.registerMonster(mySpecialMonster);
    }
}
```

Program bütün değerleri 10 olan genel bir canavarı kaydeder. Bölüm sonu canavarının tek farkı canının daha fazla olmasıdır: Registry'den genel canavar istenir, klonu alınır, canı 100 yapılır, türü "Bolum sonu canavarı" olarak adlandırılır ve daha sonra kullanılmak üzere yeniden kaydedilir.

> **Ders sonrası not:** `main` bir sınıfın içinde olsa da `setHP` ve `setType` protected'dır; `MainApp` `dp.prototype.example` paketinde değilse bu çağrılar derlenmez. Kod büyük olasılıkla `Monster` ile aynı pakettedir; Java'da protected üyeler aynı paketten de erişilebilir.

7.3 `GenericMonster` gerekli mi?

Hoca bu örnekte `GenericMonster` alt sınıfının gerekliliğinin tartışılabileceğini söyledi: `Monster` soyut değil somut yapılsa ve prototip doğrudan bu sınıf olsa problem yine çözülürdü. Öte yandan aynı oyun mantığını farklı oyunlarda kullanmak düşünülüyorsa (fantastik bir dünyada orklar ve elfler, bir bilim kurgu oyununda uzaylılar ve askerler, bir strateji oyununun birimleri gibi) somut alt sınıflar işe yarayabilir. Belirli tek bir oyun için yapılıyorsa somut alt sınıfa pek ihtiyaç olmayabilir. Bir kalıp gereksinime uygun biçimde değiştirilebilir: bazı kısımları silinebilir ya da değiştirilebilir.

8. Singleton Kalıbı

8.1 Amaç ve örnekler

Singleton'ın amacı adından anlaşılır: bir sınıfın yalnız tek bir örneğinin bulunmasını sağlamak. Başka bir deyişle bir sınıftan yalnız tek bir nesne oluşturulsun, başka bir kopyası oluşturulamasın ve bütün istemciler bu nesneye ortak bir noktadan erişsin. Normalde bir sınıfın kurucusu çağrılarak istenildiği kadar nesne oluşturulabilir; burada bu istenmez.

Hocanın örnekleri: bir bilgisayara istenildiği kadar yazıcı eklenebilir, ama o işletim sisteminde yalnız bir yazdırma kuyruğu (print spooler) süreci çalışır. Bir işletim sisteminde tek bir dosya sistemi uygulaması (Windows'taki Explorer gibi), tek bir pencere yöneticisi ve tek bir çekirdek süreci olur; başka bir çekirdek aynı makinede çalışamaz (ana bilgisayarlar ayrı bir konudur).

8.2 Static üye alan yaklaşımı ve sakıncası

Nesneye yönelimde static kavramı vardır: ortak kullanılan üye alanlar ve metotlar static tanımlanır. O hâlde tek olacak nesne static bir üye alan olarak tanımlanabilir. Ama bu nesne farklı sınıflar tarafından kullanılacaksa hangisinin üyesi olacaktır? "Herhangi birinin olsun" denirse Feature Envy kod kusuru ortaya çıkabilir. Örneğin nesne 10 farklı sınıf tarafından kullanılıyor, bir sınıf onun bir metodunu 50 kez, bir başkası 10 kez, bir başkası yalnız 1 kez çağırıyor olsun; static üye yalnız 1 kez çağıran sınıfa konmuş olabilir. Bu fark edilince Move Field yapılarak üye alan başka sınıfa taşınır. Bu çok büyük bir sorun değildir, Eclipse gibi geliştirme ortamları bunu kolayca yapar; ama ilgili getter ve setter metotlarını, sahiplik ilişkilerini de değiştirmek gerekebilir ve bu işi zorlaştırır.

8.3 Önerilen çözüm

Dörtlü Çete'nin önerdiği çözümün özü, tek olma sorumluluğunu nesnenin kendisine atamaktır. Slayttaki maddeler (ekrandan, 02:45:30):

- Bir sınıfa, kendisinin tek bir örneğinin olmasını sağlama sorumluluğunu atamak:
  - Sınıfın kendi türünden static bir üyesi olur,
  - Sınıfın kurucusu private tanımlanarak kurucuya erişim engellenir,
  - Sınıfın bu üye için bir factory metodu bulunur.
- Kalıba adını veren bu sınıfın adı Singleton olarak belirlenmiştir.

Sınıf şeması metne dökülünce:

```
Singleton
  - uniqueInstance : Singleton {static}
  - state : SingletonData
  - Singleton( ... )
  + getInstance( ) : Singleton {static}
  + singletonOperation( )
  + getterMethods( )

getInstance() notu:
  if( uniqueInstance == null ) {
      uniqueInstance = new Singleton( ... )
  }
  return uniqueInstance;
```

Kurucu private olduğu için başka hiçbir sınıf `new` ile bu nesnenin bir kopyasını oluşturamaz. Tek erişim noktası olarak paylaşılan örneği döndüren bir metot eklenir. Hoca bu metodun Factory Method kalıbı anlamında bir fabrika metodu olmadığını, paylaşılan örneği döndüren bir getter gibi düşünülmesi gerektiğini vurguladı. Adının `getInstance()` konması daha iyidir; böylece fabrika kalıplarıyla karışmaz. Static nesne üzerinde çalışan bir metodun da static olması en mantıklısıdır.

`getInstance()` örnek daha önce ilklendirilmemişse onu ilklendirir, sonra döndürür. `if` koşulu ilklendirmenin yalnız bir kez yapılmasını sağlar: daha önce ilklendirilmişse nesne değiştirilmeden yine aynı örnek döndürülür ve tekrar tekrar kopya oluşmaz.

Gerçek bir sistemde sınıfın adı Singleton olmayabilir, örneğin `Kernel` olabilir. Bu sınıfın Singleton olarak tasarlandığı belirtilmek isteniyorsa bir UML not bileşenine "Singleton'dır" diye yazılabilir; böylece tek örnekli olduğu bir bakışta anlaşılır.

Şemadaki `state : SingletonData` alanı için hoca, istenirse durum verisinin ayrı bir sınıf olarak verilebileceğini, ama genelde buna gerek olmadığını söyledi: Singleton'ın primitif ya da primitif olmayan bütün üye alanları sınıfa ayrı ayrı eklenir; aksi hâlde yapı biraz Memento'ya kayar. Kurucu, durumla ilgili bütün üye alanların bilgisini alır ama private'tır. Sınıfın gerektirdiği işlemler public metotlar olarak eklenir. Getter'lar eklenir; setter eklenip eklenmeyeceği ise gereksinime bağlıdır: nesne tek olduğu için o tek nesne set edilebilir ve herkes onu kullanır.

8.4 Gerçekleme ayrıntıları ve kullanım

Slayttaki maddeler (ekrandan, 02:50:00):

- Gerçekleme ayrıntıları:
  - Strategy ve State gerçeklemeleri aynı zamanda iyi birer Singleton adayıdır (davranışsal kalıplar ileride incelenecek).
  - Bu kalıpta Singleton sınıflarını kalıtım yolu ile özelleştirmek, ilk önerilen çözüme göre daha kolaydır.
  - Ancak Singleton nesnesi farklı türden nesneler tarafından kullanılmayacaksa, ilk öneriyi kullanmak yeterli olacaktır.
- Kalıbın kullanılabileceği anlar:
  - Bazı nesnelerin türünün tek örneği olmasının gerekli olduğu ve bu nesnelerin bir çok farklı sınıf örneklerince kullanılması istendiği anlarda.

Hoca, tek paylaşılan nesne onu kullanan sınıflardan birinin static üyesi yapılırsa, bu nesnenin alt sınıflarını oluşturmanın, sürdürmenin ve farklı uygulamalarda farklı alt sınıflar üzerinden tasarım yapmanın daha zor olacağını söyledi; Singleton'ın bu yönde bir avantajı vardır. Nesne yalnız tek bir sınıf tarafından kullanılacaksa kalıba hiç gerek yoktur; normal bir static üye alan yeterlidir. Kalıp basit olduğu ve sınıf şeması kendini anlattığı için bir örnek gerçekleme yapılmadı. Bu kalıpla birlikte yaratımsal kalıplar tamamlandı.

Örnek Sorular

1. (derste sözlü, 01:05:41) Builder'da da sonunda bir ürün nesnesi ortaya çıkıyor. Builder'ın Abstract Factory ve Factory Method'dan farkı nedir?

> **Çözüm (derste verildi):** Builder'da ürün farklı alt parçalardan oluşur; farklı ürünlerin farklı alt parçaları olabilir ve bunlar bir aile oluşturmaz. Ayrıca Builder, ürün oluşturma süreci üzerinde daha fazla denetim sağlar: Director, Builder'a hangi parçayı hangi sırada oluşturacağını adım adım söyler. Abstract Factory ve Factory Method'da istemci yalnız "şu türden bir ürün istiyorum" der, ürünün nasıl oluşturulacağına müdahale edemez.

2. (ekrandan, 01:10:00; derste sözlü, 01:10:59) RTF okuyucu örneğine yeni bir hedef biçim olarak PDF eklemek istiyoruz. Hangi sınıflar değişir?

> **Çözüm (derste verildi):** Yalnız `TextConverter`'ın alt sınıfı olarak yeni bir `PDFConverter` sınıfı yazılır ve PDF'e çevirmeyle ilgili bütün işlemler bu sınıfa kodlanır. `RTFReader`'a, içindeki switch-case'e ve kardeş çevirici sınıflara dokunulmaz. Bu, "ürünün iç yapısı en az yan etkiyle değiştirilebilir" sonucunun örneğidir.

3. (ekrandan, 01:28:10; derste çalıştırıldı, 01:32:43) `new HondaBuilder("Civic Sedan")` ve `{"Engine\t125PS","Engine\t1.6L","Gear\t6AT"}` öğeleriyle `construct` ve `report` çağrıldığında aracın özellikleri ne olur?

> **Çözüm (derste verildi):** Kasa: sedan. Güç: "PS"den önceki 125, yani 125 beygir. Hacim: 1.6 L tam sayı olarak 1600 cc. Vites: "AT" otomatik, 6 ileri. Yakıt: benzin. Program çıktısı bunu doğruladı.

4. (derste sözlü, 01:56:30) Prototype örneğinde `GraphicTool` ile `Graphic` arasında parça-bütün ilişkisi var, ama `RotateTool` ile `Graphic` arasında yok. Neden? Bu ilişki neden soyut `Tool` sınıfına taşınmamış?

Çözüm (derste verildi): Döndürme, şeklin türünü bilmeden yapılabilen ortak bir işlemdir (grafiği bir döndürme matrisiyle çarpmak). `GraphicTool`'un yaptığı değişiklikler ise şeklin kendisine bağlıdır (içi dolu ya da boş çizim, metnin italik yapılması, Bezier eğrisinin denetim noktaları). İlişki bütün kardeş alt sınıflarda olsaydı üst sınıf `Tool`'a taşınması en doğrusu olurdu; yalnız bir alt sınıfta olduğu için o alt sınıfta kalır.

5. (derste sözlü, 02:05:10) Müzik besteleme problemi Abstract Factory ya da Factory Method ile de çözülebilir mi? Dörtlü Çete neden Prototype'ı seçmiştir?

Çözüm (derste verildi): Çözülebilir. Abstract Factory'de nota türleri (tam, yarım, ters) aile, do'dan si'ye notalar ailenin üyeleri olur; aileler ve üyeleri için daha çok sınıf gerekir. Factory Method'da her nota türü için ayrı bir fabrika sınıfı gerekir ve ürün hiyerarşisine paralel bir fabrika hiyerarşisi oluşur. Prototype ise yalnız nota türü kadar sınıfla, daha basit biçimde bütün ihtiyaçları karşılar.

6. (derste sözlü, 02:20:31) Java ve Dart derlenen dillerdir. Yansıtma (reflection) ile bir nesnenin yapısını çalışma anında sorgulamak, hatta değiştirmek nasıl mümkün olur?

Çözüm (derste verildi): Hocanın yanıtına göre bu, iki dilde de bir çalışma ortamı (sanal makine) bulunması sayesinde mümkündür. Java'da sınıf kodu belleğe gerektiğinde sınıf yükleyici tarafından yüklenir ve bu bileşen sanal makineyle birlikte değiştirilebilir. Böyle bir çalışma ortamı olmayan dillerde bu iş ya mümkün değildir ya da çok daha zordur. (Java'nın bytecode'a derlenip JVM'de çalıştırıldığına ilişkin düzeltme için 6.8'deki ders sonrası nota bakın.)

7. (ekrandan, 02:20:00) Slayttaki "Ürün sınıfının örneklerinin durum uzayı sınırlı ise, bu durumda sınıfın iyi tasarlanıp tasarlanmadığı ayrıca tartışılır" maddesi neden yazılmıştır?

Çözüm (derste verilmedi): Hoca gerekçeyi derste çıkaramadı ve daha sonra düşünmek üzere not aldı. Büyük olasılıkla anlamı şudur: Dörtlü Çete'ye göre Prototype, bir sınıfın örnekleri yalnız birkaç farklı durum birleşiminden birine sahip olabildiğinde kullanışlıdır, çünkü her birleşim için bir prototip kurulup klonlanır. Ancak örnekler yalnız birkaç sabit durumda bulunuyorsa, bu durumların ayrı alt sınıflarla ya da bir enum gibi başka bir yapıyla modellenmesinin daha uygun olup olmadığı da sorgulanmalıdır.

Dersin Toparlaması

Bu derste yaratımsal kalıpların geri kalanı tamamlandı. Factory Method, tek bir nesnenin oluşturulmasını soyut bir fabrika metoduna bırakıp hangi sınıfın oluşturulacağına alt sınıfların karar vermesini sağlar; belge çerçevesi, çizim uygulaması ve e-ticaret örnekleriyle işlendi ve ürün ailesiyle çalışan Abstract Factory'den farkı vurgulandı. Builder, parçalardan oluşan karmaşık bir nesnenin oluşturulmasını gösteriminden ayırır; Director'ın oluşturma sürecini adım adım yönetmesiyle fabrika kalıplarından daha fazla denetim sağlar, ama switch-case'in ConcreteBuilder'a kayması ve boş gövdeli metotlar gibi zayıf yönleri vardır. RTF çevirici ve araç adı çözümleyici örnekleri bunu somutlaştırdı.

Prototype, yeni nesneleri bir prototipi klonlayarak oluşturur; çalışma anında belirlenen sınıflar, paralel fabrika hiyerarşisinden kaçınma ve daha az sınıfla çözüm onun Factory Method'a üstünlükleridir. Sığ ve derin kopyalama ayrımı, çevrimli ilişkilerin riski ve yansıtma ya da geç ilişkilendirme sunan dillerde kalıba gerek kalmayabileceği tartışıldı. Son olarak Singleton ile bir sınıfın tek örneğinin bulunması, tek olma sorumluluğunun sınıfın kendisine verilmesiyle (static üye, private kurucu, `getInstance()`) sağlandı. Hoca Strategy ve State gerçeklemelerinin iyi birer Singleton adayı olduğunu, bu davranışsal kalıpların ileride inceleneceğini belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.